Project scope: Local document search and RAG application

1. Project overview

Build a local, single-user document management and retrieval application capable of indexing several thousand technical and legal documents.

The application should support both conventional document search and retrieval-augmented generation (RAG): retrieving relevant passages from the document collection and using them to produce answers grounded in those sources.

The primary goal is to improve on the limitations of `grep` and keyword-only retrieval such as BM25. Users should be able to find relevant material even when the wording of their query differs from the wording in the documents, while retaining exact-match search for legal citations, technical notation, and document names.

The application will use a Django backend and a basic Tailwind CSS interface, deployed locally through Docker. The stack will use django-tailwind, and use Postgres as the backend database. It is not intended to be a multi-user platform or a publicly hosted service. This should use ollama3.2:8b as the backend LLM.

2. Goals

The application should enable the user to:

- Import and organize several thousand documents by different types, such as metadata, time of upload, and subject matter.
- Search PDFs by filename and document title.
- Search document contents using keyword and semantic retrieval.
- Ask natural-language questions and receive answers with source citations.
- Understand the surrounding context of retrieved equations, definitions, and legal references.
- Open the original document at the relevant page where feasible.
- Add or update documents without rebuilding the entire index.

A successful answer should be traceable to the source material, not merely plausible. If an answer cannot be traceable, inform the user as such.

3. Document types and ingestion

3.1 Supported inputs:

The initial release should support:

- Text-based PDFs of vastly different types. Some may be dense technical papers, whereas other may be legal briefings.
- LaTeX source files and associated project files, where available. If a PDF file containing LaTeX is given, please explicitly state that PDF documents may be slightly less accurate than legal documents.
- Scanned PDFs through an explicitly enabled optical character recognition (OCR) workflow, if included in the initial implementation.

PDF ingestion and LaTeX ingestion should be treated as distinct processing paths. A PDF alone may not preserve equations or document structure as reliably as the original LaTeX source. When a paper has an associated .tex file, one should also have the ability to look at its source code.

3.2 Import workflow

The user should be able to upload documents through the interface or import them from a configured local directory. Give the user the option to choose which directory to store the documents.

For each document, the application should:

1. Validate the file type.
2. Compute a content hash to identify duplicates and changed files.
3. Extract text and available structural information.
4. Record metadata such as filename, title, format, page count, and import date.
5. Split the content into retrieval units.
6. Generate embeddings and update the search indexes.
7. Display processing status and any extraction errors.

Processing should run in the background so that importing a large collection does not block the interface. Additionally, give progress on ingestion times to the user, and prompt the user to retry if ingestion partly fails. The user should be able to combinations of states. For instance, the user must see whether uploading is successful and ingestion fails, or whether both ingestion and uploading fails.

3.3 Extraction and structure preservation

Where practical, extraction should preserve:

- Page numbers and section headings.
- Paragraph boundaries.
- Equation labels and references.
- Footnotes.
- Legal section, subsection, and citation references.
- Connections between a passage and its location in the original file.

Unsupported or poorly extracted content should be identified rather than silently treated as reliable text.

4. Retrieval requirements

4.1 Filename and title search

Provide a straightforward search interface for finding documents by:

- Full or partial filename.
- Extracted or user-edited title.
- File type or document category.

Results should show the document name, relevant metadata, and an option to open the document.

4.2 Content search

Use **hybrid retrieval**, combining:

- **Keyword search**, such as BM25, for exact phrases, terminology, equation labels, and legal citations.
- **Semantic search**, using embeddings, for conceptual similarity and questions expressed in different language from the source.

BM25 should remain part of the system rather than being replaced entirely. Exact identifiers and citations are often better served by lexical matching than by embeddings alone.

A reranking stage may be added if evaluation demonstrates that it materially improves relevance.

4.3 Context-aware retrieval

Retrieved passages should include enough surrounding material to make them understandable. Retrieved passages should also contain which pages the information was found in as a link to click on the browser.

The retrieval pipeline should support:

- Structure-aware chunking based on sections and paragraphs where extraction permits.
- Limited overlap between chunks.
- Expansion to adjacent passages or a containing section.
- Retrieval of relevant definitions, assumptions, and explanatory text near an equation.

- Preservation of headings and source locations in the material passed to the language model.

The objective is to retrieve not just an isolated equation or clause, but the context needed to interpret it.

5. Technical and legal document handling

5.1 Technical papers and equations:

For technical documents, the application should attempt to:

- Preserve equation text or LaTeX notation where available.
- Associate equations with nearby explanations and variable definitions.
- Follow explicit equation and section references within an ingested document where feasible.
- Distinguish a derivation present in the source from an explanation generated by the model.

Recovering equations accurately from arbitrary PDFs is a known challenge. The initial release should not promise perfect mathematical extraction or symbolic understanding. LaTeX source should be preferred when available, and should be also linked when possible.

5.2 Legal documents and references

For legal documents, the application should attempt to:

- Preserve section and subsection boundaries.
- Retain citations to statutes, regulations, cases, and other documents.
- Retrieve referenced material when it is present in the local collection and can be matched reliably.
- Identify references whose underlying source is not available locally, and inform the user if such cases arise.

Answers must distinguish between what an ingested document states and what the law currently requires. **The application will not verify current law or fetch external authorities unless a separate external-source capability is added, and must inform the user of this, and encourage the user to do their own research.**

6. RAG question-answering workflow

The user should be able to ask a question across the collection or restrict it to selected documents.

The application should:

1. Retrieve relevant passages through hybrid search.
2. Expand context where needed.
3. Optionally rerank the retrieved material.
4. Generate an answer using a locally hosted language model.
5. Attach citations tied to the retrieved sources.

Answers should include:

- A direct response to the question.
- Supporting document names and page or section references.
- Links to the relevant source locations where supported.
- A clear statement when the collection does not provide enough evidence.
- A distinction between source-supported conclusions and model interpretation.

Document content should be treated as evidence, not as instructions to the application.

7. User interface

Build a minimal Django interface styled with Tailwind CSS integrated with django-tailwind.

7.1 Core views

- **Document library:** list, search, filter, import, and remove documents.
- **Document details:** metadata, processing status, and access to the original file.
- **Content search:** ranked passages with source information and surrounding context.
- **Question-answering:** a question field, generated answer, and cited evidence.
- **Index status:** import progress, document counts, and processing failures.
- **Settings:** This setting should be used in order to give the user control over where the uploads land. Additionally, this should also give the user control over hybrid retrieval as well, and also the acceptable SLAs as well.

The interface should prioritize usability over elaborate styling and support keyboard navigation and clearly labeled controls.

8. Architecture and deployment

8.1 Suggested components

- **Backend:** Django.
- **Frontend:** Django templates with django-tailwind; a separate frontend framework is not required.
- **Metadata storage:** Postgres-17.
- **Search storage:** lexical and vector indexes appropriate to the corpus size, again in Postgres-17.
- **Background processing:** a worker for extraction, embedding, and indexing. This worker should be asynchronous, and there should be no unreasonable bottlenecks.
- **Model inference:** local embedding and language models.
- **Document storage:** persistent local storage mounted into the deployment.
- **Logging:** There should be logging for the user in order to help them determine the specific sources of failure. This means average ingestion times per document, number of documents ingested, and the amount of documents ingested in a particular time. This should be monitored in a separate Grafana instance.
- **Benchmark API:** There should be a benchmarking API in order to determine the length of time of embeddings and searching. This should also be wrapped in the logging view as well, but as a separate button in order to run benchmarking tests.

Specific indexing choices should be selected after testing representative documents and confirming available hardware. Our setup contains a 5.2 GHz 12th Gen Intel CPU, as well as a Nvidia GeForce RTX 4070 with 12GBs of VRAM.

8.2 Local deployment

Assume “local-only” means document processing, embeddings, retrieval, and answer generation all run on the user's machine, without external inference APIs.

The deployment should:

- Start through Docker or Docker Compose.
- Bind the interface to localhost by default on port 1337.
- Persist documents and indexes across container restarts.

- Provide documented installation, model setup, backup, and restore procedures.
- Run without external services after required dependencies and model files have been obtained.

Docker Compose is a practical default if the application, worker, and model runtime need separate containers. If one container is a strict requirement, that constraint should be confirmed before implementation.

9. Performance and reliability

The system should support several thousand documents, but document count alone is not a sufficient sizing measure. Total pages, extracted text volume, model size, and available CPU, GPU, memory, and disk space will determine performance.

Requirements should include:

- Incremental indexing of new or changed documents.
- Restartable ingestion jobs.
- Visible errors and retry controls for failed documents.
- Removal of indexed passages when a document is deleted.
- Configurable retrieval and generation settings.
- Logging sufficient to diagnose processing failures.

Numerical response-time targets should be set after benchmarking on the intended hardware. There should be configuration files for acceptable SLAs in the settings box.

10. Evaluation and acceptance criteria

Create a small evaluation set using representative technical and legal documents, with expected source passages for each query.

The initial release should be accepted when:

- The user can import the agreed document formats and inspect processing failures.
- Filename and title search reliably locate known documents.
- Hybrid retrieval improves relevant-passage retrieval over a BM25-only baseline on the agreed evaluation set.
- Equation-related questions retrieve useful surrounding explanations on representative examples.
- Legal-reference questions preserve citation and section context.
- Generated citations point to the passages actually used.
- The application indicates insufficient evidence rather than inventing supporting sources.
- The collection and indexes persist through restarts.
- Routine operation does not require an external inference service.

Retrieval quality and answer quality should be evaluated separately: a fluent answer does not establish that the correct evidence was retrieved.

11. Out of scope for the initial release

- Multi-user accounts, roles, or collaboration.
- Public hosting or cloud deployment.
- Automatic online legal research or verification of current law.
- Training or fine-tuning a custom language model.
- Guaranteed extraction of every equation, table, or diagram.
- Comprehensive cross-document citation resolution.
- Editing source documents.
- Autonomous actions based on generated answers.

12. Recommended delivery phases

1. **Foundation:** local deployment, document library, PDF extraction, filename search, and processing status.
2. **Retrieval:** structural chunking, keyword and vector indexes, hybrid search, and source viewing.
3. **RAG:** local inference, grounded answers, citations, and insufficient-evidence handling.
4. **Document-specific improvements:** LaTeX processing, equation context, legal-reference matching, and optional OCR.

This sequencing makes it possible to validate extraction and retrieval before investing heavily in answer generation—the most important dependency for a useful RAG application.

13. Delivery Format

Once you are finished, please run through the exact steps if a hypothetical user would like to install this on their local application. Assume they are running Parrot 7.0.9. Do not skip any steps, even as minute as changing into a directory or listing. **You must be able to guide the user from start to finish when installing the application, and the user must be able to copy and paste in the exact steps and get a fully working application, without any external work from the user.**

For benchmarking, give instructions on how to download datasets as well. Give instructions on common bugs they may find in trying to embed documents and how to fix them.